LAB 3

TITLE: Implementation of Shortest Job First (SJF) CPU Scheduling Algorithm in C

OBJECTIVE:

To implement the Shortest Job First (SJF) CPU scheduling algorithm in C and calculate:

- Waiting Time (WT)
- Turnaround Time (TAT)
- Average Waiting Time
- Average Turnaround Time

THEORY:

Shortest Job First (SJF) is a **non-preemptive** scheduling algorithm where the process with the shortest burst time is executed first. This improves the average waiting time compared to FCFS.

➢ **Characteristics:**

- It is based on burst time, not arrival time (in non-preemptive).
- Shorter jobs are prioritized, which reduces waiting time.
- If burst time is not known in advance, it can be difficult to implement practically.

➢ **Terminologies:**

- Burst Time (BT): Time needed for process execution.
- Waiting Time (WT): Time a process waits before execution.
  - WT[i] = ∑ BT of all previous sorted processes
- Turnaround Time (TAT): Total time from arrival to completion.
  - TAT[i] = WT[i] + BT[i]
- Average Waiting Time (AWT):
  - AWT = ∑WT / n
- Average Turnaround Time (ATAT):
  - ATAT = ∑TAT / n

Algorithm:

1. Start
2. Input number of processes n
3. Input burst time for each process
4. Assign process IDs
5. Sort processes by burst time in ascending order (SJF logic)
6. Set waiting time of first process = 0
7. For remaining processes:
   - WT[i] = WT[i-1] + BT[i-1]
   - TAT[i] = WT[i] + BT[i]
8. Calculate average WT and TAT
9. Display all data
10. End

PROGRAMS

```
#include <stdio.h>

// Structure to represent a process
struct Process {
    int pid;        // Process ID
    int burst;      // Burst time
    int waiting;    // Waiting time
    int turnaround; // Turnaround time
};
int main() {
    int n, i, j;
    float avg_waiting = 0, avg_turnaround = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process proc[n], temp;

    // Input burst times for each process
    for(i = 0; i < n; i++) {
        printf("Enter burst time for process P%d: ", i
+ 1);
```

```
        scanf("%d", &proc[i].burst);
        proc[i].pid = i + 1;  // Assign process IDs
    }

    // Sort processes by burst time (SJF)
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(proc[j].burst > proc[j + 1].burst) {
                // Swap processes
                temp = proc[j];
                proc[j] = proc[j + 1];
                proc[j + 1] = temp;
            }
        }
    }

    // Calculate waiting and turnaround times
    proc[0].waiting = 0;  // First process has 0 waiting time
    proc[0].turnaround = proc[0].burst;

    for(i = 1; i < n; i++) {
        proc[i].waiting = proc[i - 1].turnaround;
        proc[i].turnaround = proc[i].waiting + proc[i].burst;
    }

    // Calculate averages
    for(i = 0; i < n; i++) {
        avg_waiting += proc[i].waiting;
        avg_turnaround += proc[i].turnaround;
    }

    avg_waiting /= n;
    avg_turnaround /= n;

    // Print results
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\n",
               proc[i].pid, proc[i].burst, proc[i].waiting, proc[i].turnaround);
    }

    printf("\nAverage Waiting Time: %.2f", avg_waiting);
    printf(" ms\nAverage Turnaround Time: %.2f ms\n", avg_turnaround);

    return 0;
}
```

Output:

```
Enter number of processes: 4
Enter burst time for process P1: 6
Enter burst time for process P2: 8
Enter burst time for process P3: 7
Enter burst time for process P4: 3

Process Burst Time   Waiting Time   Turnaround Time
P4      3            0              3
P1      6            3              9
P3      7            9              16
P2      8            16             24

Average Waiting Time: 7.00 ms
Average Turnaround Time: 13.00 ms
```

RESULTS:

The **Shortest Job First (SJF)** scheduling algorithm was successfully implemented. The program accurately calculated the **waiting time**, **turnaround time**, and their averages based on burst time sorting.

CONCLUSION:

SJF scheduling reduces the average waiting time compared to FCFS. This lab provided understanding of:

- Sorting processes by burst time
- Calculating efficient process scheduling
- Trade-off between fairness and efficiency in scheduling algorithms